

Message-Passing Graph Neural Network as a Surrogate for Variable-Length 1D Finite Element Bead Chain Simulations

Grey L. Goodwin | Daniel L. Lau, PhD.

Department of Electrical and Computer Engineering · University of Kentucky · grey.goodwin@uky.edu · dllau@uky.edu

Abstract

Finite element analysis (FEA) of a flexible bead chain requires thousands of small, localized force and integration computations per simulated second. We train a graph neural network (GNN) to predict the per-timestep physics of a 1D bead chain with tapered mass, using randomly sampled single-step training data instead of recorded simulations. A key finding was that even a well-designed model will fail without the correct training data. Sampling neighbor positions in polar coordinates near the rest length was necessary for stable rollouts, while uniform Cartesian sampling produced models that diverged. With this data pipeline in place, an initial multilayer perceptron (MLP) achieved accurate predictions on a fixed 16-bead chain but would require further training for a chain of a different length. We address this by restructuring the network as a true message-passing GNN with separate encoders for node and edge features, ghost masking for chain endpoints, and weight sharing across all nodes. The same trained model runs on chains of any length. We train on 1M per-bead samples drawn from chains of 8 to 32 beads with random taper ratios. Once trained, we were able to evaluate rollout accuracy, constraint satisfaction, and generalization on chain lengths never seen during training. The model maintains accurate performance across the trained range and handles varied initial orientations and velocities. We discuss the connection between the local message-passing structure and the underlying physics, the relationship between sampling strategy and generalization, and remaining limitations.

Problem Setup

- Chain of N mass nodes connected by $N-1$ spring rod elements.
- Tapered mass distribution from heavy anchor to light tip (ratio $r = m_0/m_{N-1}$).
- Forces per bead: gravity, Hooke's-law spring forces from each connected neighbor, viscous rod damping, and a small global velocity drag.
- Reference solver uses symplectic Euler integration ($\Delta t = 1e-4$ s).
- Physics is strictly local: force on bead i depends only on i and its immediate neighbors $i-1$, $i+1$.

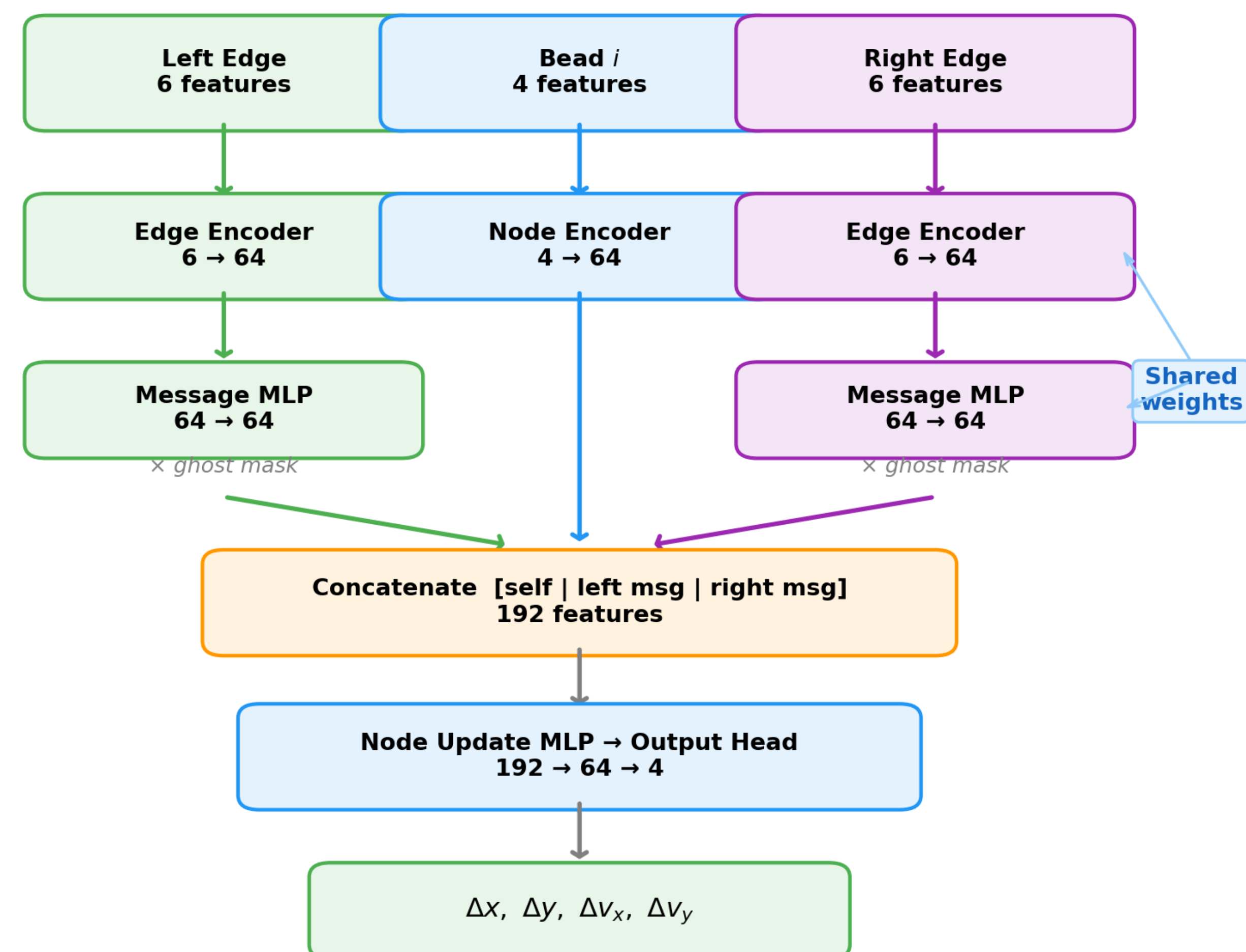
MPGNN Architecture

Five learned components, all weight-shared across beads:

- Node encoder $\rightarrow$ 64-d hidden from 4-d node features
- Edge encoder $\rightarrow$ 64-d hidden from 6-d rel. edge features
- Message MLP $\rightarrow$ per-edge 64-d message
- Node update $\rightarrow$ $[h_{\text{self}} \parallel m_{\text{left}} \parallel m_{\text{right}}] \rightarrow$ 64-d
- Output head $\rightarrow$ 4-d Δ state (Δx , Δy , Δv_x , Δv_y)

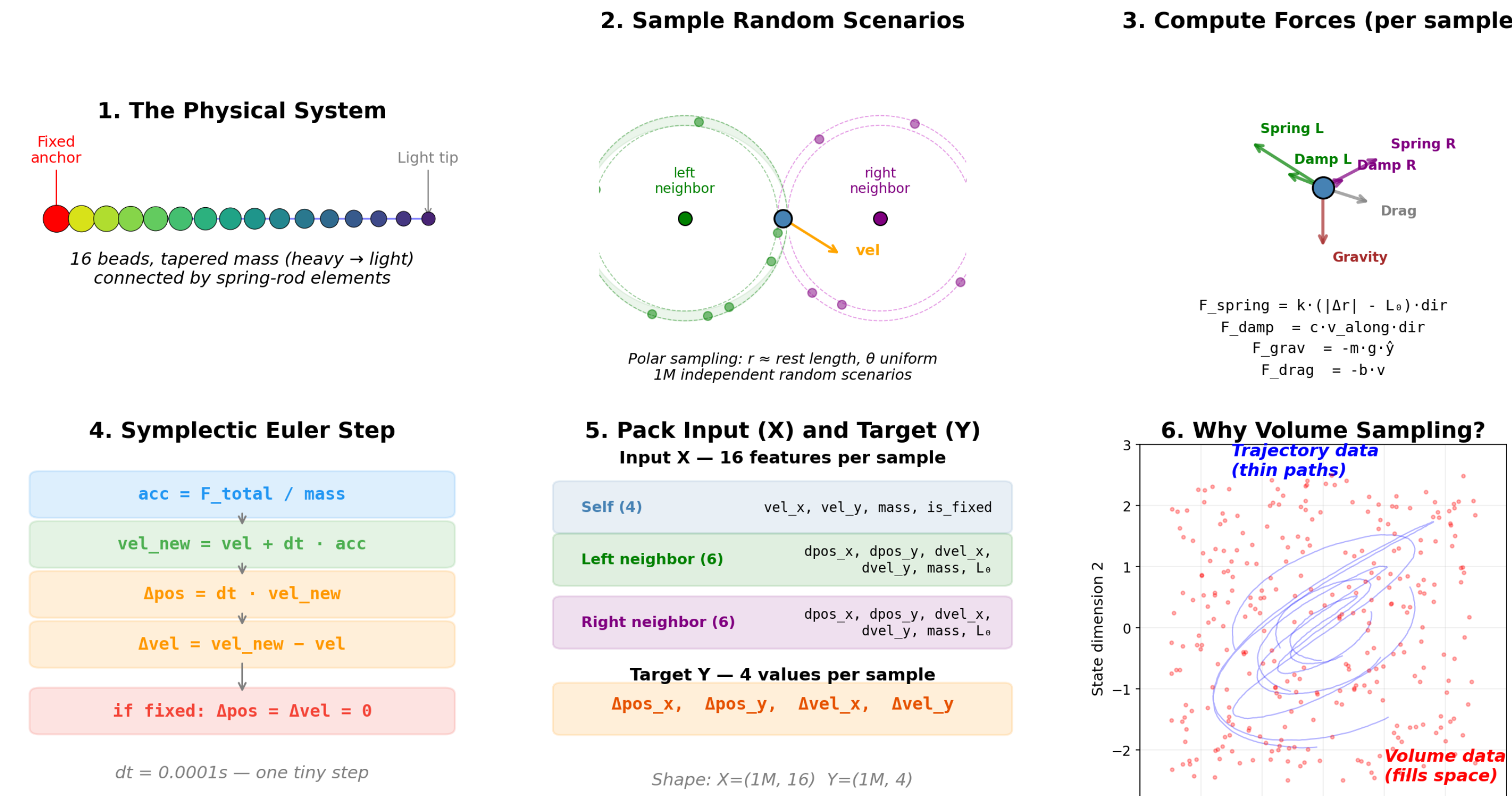
Ghost masking zeros messages from absent endpoint neighbors. 80,324 params; the same model runs on any chain length.

BeadMPGNN



Volume-Sampled Training Pipeline

generate_volume_data.py — Pipeline Visualization



Each sample is an independent random bead-level scenario: random $N \in [8, 32]$, taper ratio $r \in [1, 10]$, polar-sampled neighbors near rest length. Physics solved analytically; no trajectories recorded. Dataset: 1,000,000 samples.

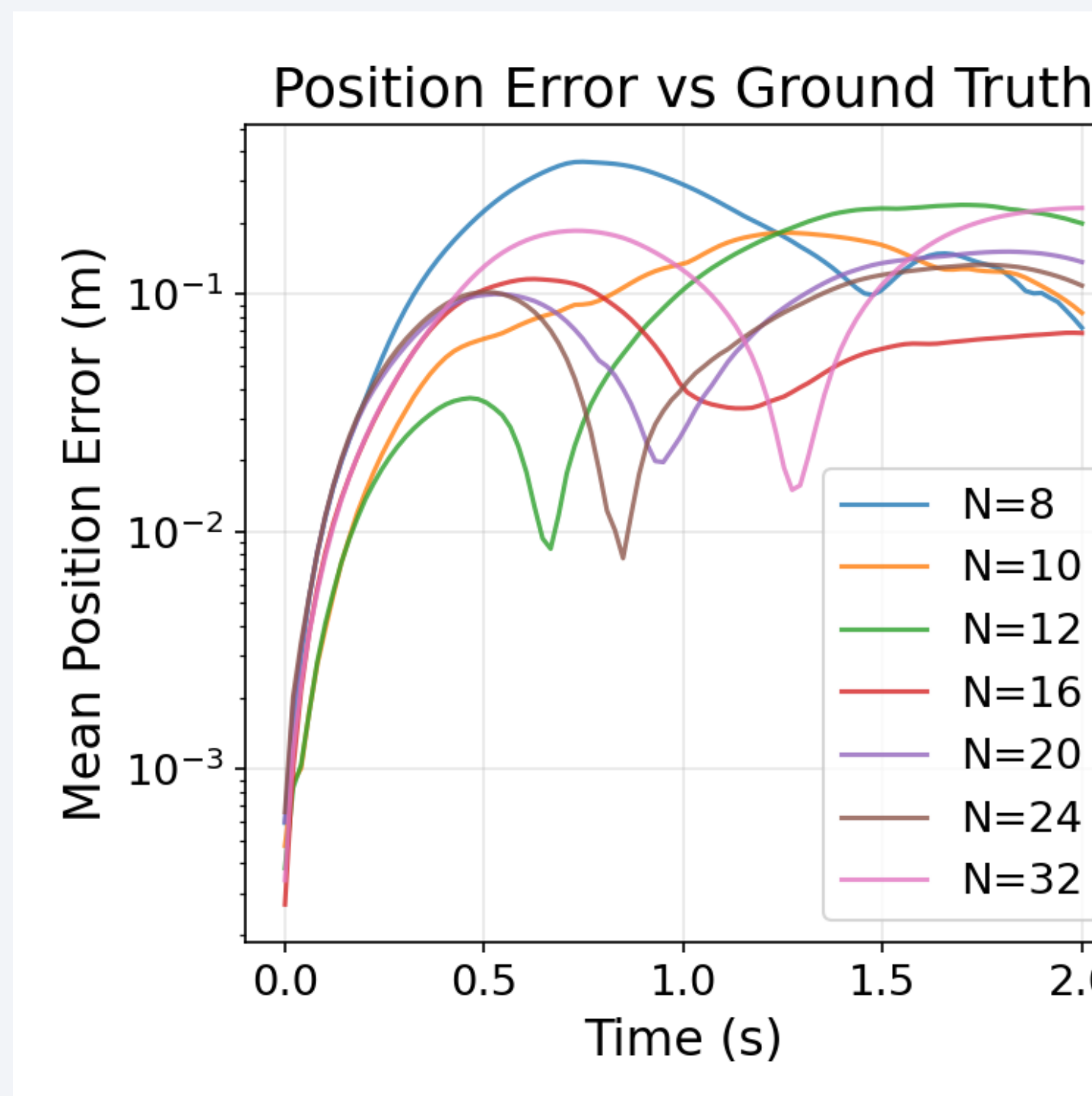
Why Volume Sampling

Trajectory-recorded data traces a thin path through state space. During rollout, small prediction errors push the chain off this path into states the model has never seen — errors compound and the simulation diverges.

Volume sampling fills the reachable state space uniformly. Every training sample has a physically realistic rod length ($\pm 5\%$ of ℓ_0), so the model learns force magnitudes matched to what it sees at inference.

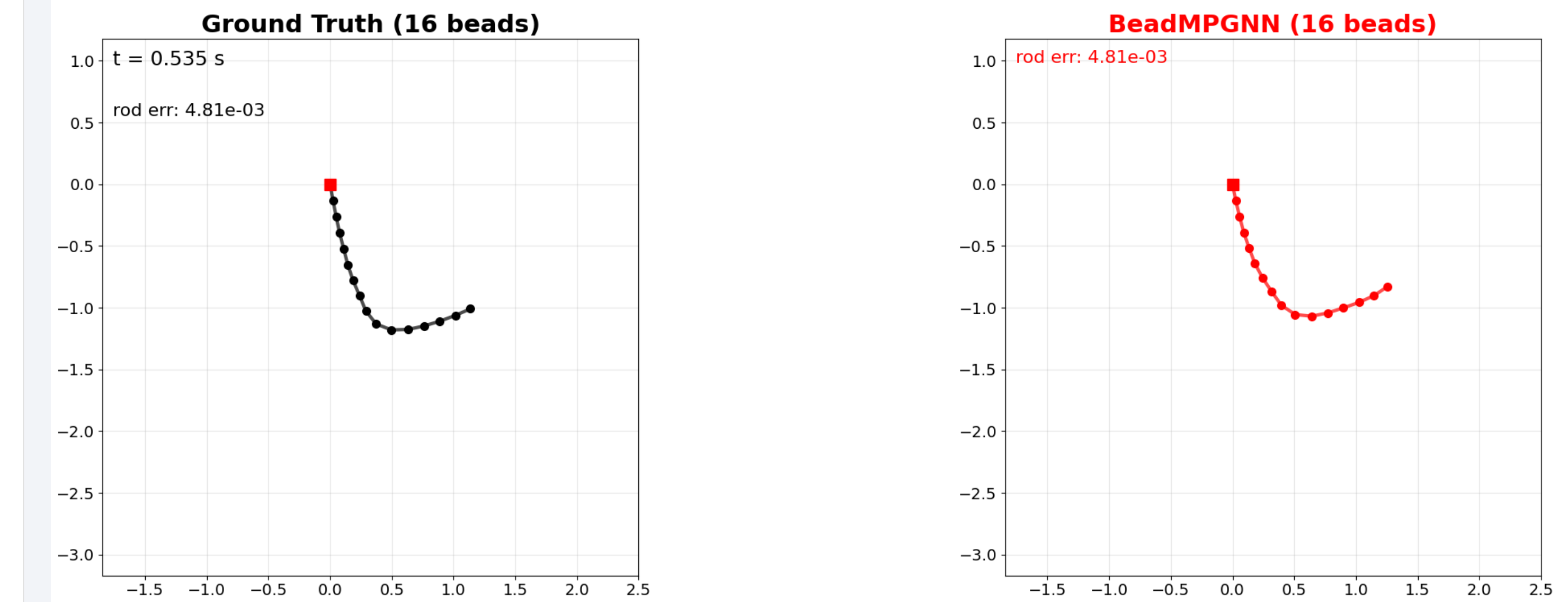
- Cartesian uniform sampling $\rightarrow$ stable loss, diverging rollouts
 - Polar near rest length $\rightarrow$ stable, accurate rollouts
- Sampling strategy matters as much as architecture.

Generalization Across N



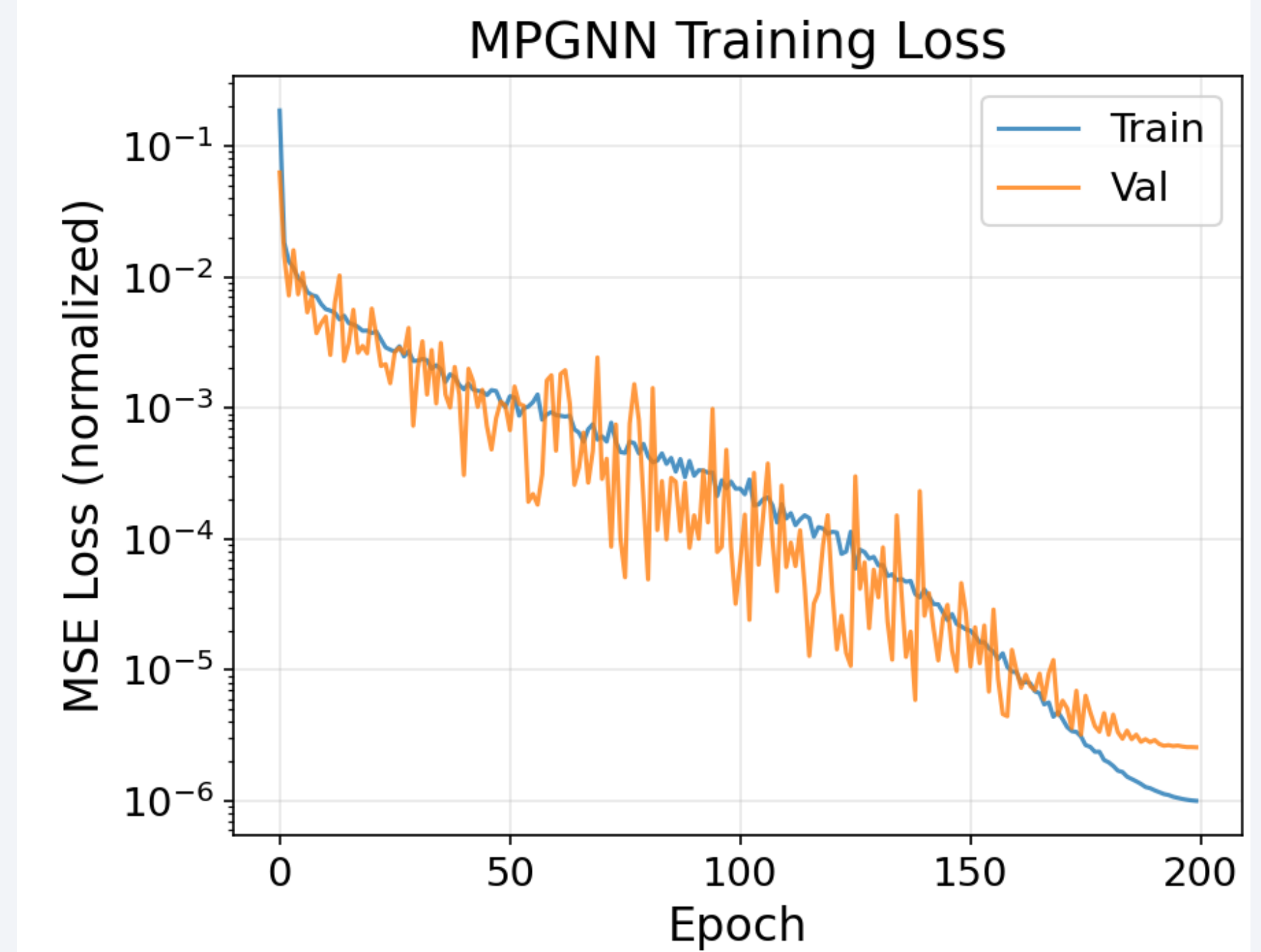
Mean position error vs. reference for $N \in \{8, 10, 12, 16, 20, 24, 32\}$. Short to medium chains track closely; longer chains show expected drift from compounding per-step errors.

Side-by-Side Rollout



Reference solver (black, left) vs. BeadMPGNN (red, right) on a 16-bead chain released from horizontal. The GNN tracks the reference through fall, swing, and tip whip.

Training Convergence



Train/validation MSE over 200 epochs. AdamW, $\text{lr} = 1e-3$ with cosine anneal, batch 512, 10% validation split, gradient clipping at norm 1.0.

Conclusions

- Sampling strategy is as important as architecture. Polar sampling near rest length is required; Cartesian uniform sampling fails.
- MPGNN with weight sharing + ghost masking enables one model across chain lengths.
- Volume sampling covers reachable state space. No trajectory recording needed.
- Model generalizes to unseen N , orientations, and velocities within sampling bounds.
- Future: rollout training to reduce drift; $K > 1$ message passing for longer-range effects.

Acknowledgements

This work was supported in part by the National Science Foundation under Grants CCF 2230161 and 2230162, and in part by AFOSR under Grant FA9550-22-1-0362.